SOMNGUARD

Estrategia de CI/CD

Estado: En progreso **Fecha:** 2026-08-19

Estrategia de integración y despliegue continuo del proyecto, alineada con la metodología adoptada (`../00-documentation-governance/adopted-methodology.md`) y el documento de arquitectura.

Herramienta

- **GitHub Actions** como orquesta de CI/CD.
- **Maven** para build y ejecución de pruebas del backend (Java 21 / Spring Boot 4.1.1).
- Gestión de secretos mediante GitHub Secrets (nunca credenciales en el repositorio).

Flujo de ramas

Cada historia se despliega por ambiente con su propia rama `hu-<repo>-###-<ambiente>`. Los cambios se propagan entre ambientes copiándolos o con `cherry-pick` (nunca por merge directo entre `develop`, `qa` y `main`):

```
hu-<repo>-###-dev → develop (PR + review)
     ↓  (copiar cambios / cherry-pick)
hu-<repo>-###-qa → qa (pruebas manuales/automatizadas)
     ↓  (copiar cambios / cherry-pick)
hu-<repo>-###-main → main (producción controlada)
```

- `develop`, `qa` y `main` no se trabajan directamente: solo reciben PRs desde las ramas `hu-<repo>-###-<ambiente>`.
- Los PRs hacia `qa` y `main` deben incluir los mismos cambios ya validados en el ambiente anterior.

Pipeline propuesto

Job 1: Build (cada push a ramas `hu-*` y `develop`)

- Checkout del repositorio.
- Setup de Java 21 (Temurin).
- `mvn verify` (compila, corre pruebas unitarias e integración).
- Publicación de reportes de pruebas.

Job 2: Calidad (cada push a `develop`)

- Linters y análisis estático (Spotless/Checkstyle; SonarQube o equivalente propuesto).
- Reporte de cobertura (JaCoCo).
- Verificación de dependencias contra vulnerabilidades conocidas (OWASP Dependency-Check propuesto).

Job 3: Despliegue a `qa` (merge a rama `qa`)

- Build de artefactos.
- Despliegue del backend a entorno `qa`.
- Ejecución de pruebas E2E (Playwright web) cuando estén disponibles.

Job 4: Publicación a main (PR de `hu-<repo>-###-main` hacia `main`)

- Los cambios validados en `qa` se copian a la rama `hu-<repo>-###-main` (copiar cambios o `cherry-pick`).
- PR de `hu-<repo>-###-main` hacia `main` con checks en verde y aprobación explícita del Líder Técnico.
- Empaquetado de artefactos (`somnguard-v{MAJOR}.{MINOR}.{PATCH}.zip`).
- Publicación del release con changelog desde commits (Conventional Commits).
- Despliegue a producción controlado.

Reglas de protección

Rama	Regla
`develop`	PR con revisión (1 aprobación) y checks del pipeline en verde
`qa`	PR desde `hu-<repo>-###-qa` con validación de tester/QA
`main`	PR desde `hu-<repo>-###-main`, checks en verde y aprobación explícita del Líder Técnico

Releases

- Versionado semántico: `v{MAJOR}.{MINOR}.{PATCH}` (ej: `v1.0.0`).
- Nombres de artefactos: `somnguard-v{MAJOR}.{MINOR}.{PATCH}.zip`.
- Changelog generado desde los mensajes de commit.

Pendientes

- Definir infraestructura de despliegue (local/nube) — ver preguntas abiertas.
- Definir estrategia de contenedores (Docker) si aplica.
- Mover secretos a un Secret Manager cuando exista la capa de despliegue (hoy se manejan con `.env.*` fuera de git, ver environments.md).

Ver también

- Ambientes — ramas, BD y reglas por ambiente
- Setup local — entorno de desarrollo en local
- Plan de despliegue — plantilla de despliegue por release
- Release checklist — gates de salida de un release
- Plan de rollback — contingencia ante fallos